CS 4380 Project 4: Procedures and Beyond

Introduction

At this point in the semester, you have created a working emulator and assembler for the 4380, and have used it to investigate the implications of program behavior and caching. For this, the fourth and final project of the semester you will complete your emulator and assembler by implementing the 4380 Mark 4. The Mark 4 supports includes instructions to support procedure calls, dynamic memory management, as well as string and array manipulation. With these added features in place you will exercise the full range of your assembler and emulator's capabilities by implementing several programs that will be described below. These programs begin simple and increase in complexity, with the final program being worth 50% of your final exam grade.

Part 1: 4380 Emulator and Assembler Mark 4

To meet the requirements of this lab it will be necessary to extend the 4380 emulator and assemblers with support for the following additional instructions:

Logical Instructions: (NOT BITWISE)

Value	Operator	Operand 1	Operand 2	Description
18	AND	RD	RS	Perform RD && RS, store result in RD
19	OR	RD	RS	Perform RD || RS, store result in RD

Arithmetic Instructions:

Value	Operator	Operand 1	Operand 2	Description
14	ADDI	RD	Imm*	Add Imm to RD, store the result in RD
33	MULI	RD	Imm*	Multiply RD by Imm, store the result in RD
34	DIVI	RD	Imm*	Divide RD by Imm (signed), result in RD
38	SDIV	RD	RS	Store result of signed division RD / RS in RD

* When represented in an assembly file the Imm value shall be represented as a numeric literal.

Procedure/Stack Instructions:

Value	Operator	Operand 1	Operand 2	Description
40	PSHR	RS	DC*	Set SP = SP - 4; Place the word in RS onto the stack
41	PSHB	RS	DC*	Set SP = SP - 1; Place the least significant byte in RS onto the stack
42	POPR	RD	DC*	Place the word on top of the stack into RD, update SP = SP + 4
43	POPB	RD	DC*	Place the byte on top of the stack into RD, update SP = SP + 1
44	CALL	Address	DC*	Push PC onto stack, update PC to Address
45	RET	DC*	DC*	Pop stack into PC

*DC = "Don't Care". This operand shall be omitted in 4380 assembly instructions but is required in 4380 bytecode.

Heap Instructions:

Value	Operator	Operand 1	Operand 2	Description
35	ALCI	RD	Imm*	Allocate Imm bytes of space on the heap. The Imm value is a 4-byte unsigned integer. Store the initial heap pointer in RD.
36	ALLC	RD	Address**	Allocate a number of bytes on the heap according to the value of the 4-byte unsigned integer stored at Address. Store the initial heap pointer in RD.
37	IALLC	RD	RS	Indirectly allocate a number of bytes on the heap according to the value of the 4-byte unsigned integer at the memory address stored in RS. Store the initial heap pointer in RD.

* When represented in an assembly file the Imm value shall be represented as a numeric literal.
** When represented in an assembly file the Address shall be represented as a label in the data section.

Taps/Interrupts:

Value	Operator	Operand 1	Operand 2	Description
21	TRP	#5	DC*	Write the null-terminated Pascal-style string whose starting address is in R3 to stdout.
21	TRP	#6	DC*	Read a string from stdin and store it in memory as a null-terminated Pascal-style string whose starting address is in R3.

*DC = "Don't Care". This operand shall be omitted in 4380 assembly instructions but is required in 4380 bytecode.

Directives:

Directive	Operand	Examples	Behavior
.BTS	Required unsigned decimal value. This value is the number of bytes to be allocated.	.BTS #25 .BTS #5 a_label .BTS #20	Allocates the specified number of bytes in place and initializes them all to 0. If an optional label is present the label shall be associated with the address of the first byte allocated.
.STR	Required double quote delimited string OR a numeric literal. 255 shall be the maximum length of the string and the maximum value of the numeric literal.	.STR "Example!" name .STR "Fred" .STR #40 .STR #255	Allocates a number of bytes equal to the length of the string + 2. The first byte is initialized to the length of the string, and the last byte to 0 (null character). The remaining bytes (in the middle) are initialized to the ascii values for the characters in the string. **Note:** escape sequences such as \n must be properly handled. OR Allocates a number of bytes equal to the numeric literal + 2. The first byte is initialized to the value of the numeric literal and the remaining bytes are initialized to zeros. In both cases if an optional label is present the label shall be associated with the address of the first byte allocated.

Unless otherwise specified below - all requirements from projects 1 , 2, and 3 remain in force for the assembler and emulator. The following additional requirements must also be met.

Req1) Project 4 work must be completed in a separate branch called project-4. When all requirements are met, the branch should be merged into main (BUT NOT DELETED).

Req2) Upon startup, and after reading the input file into prog_mem, the 4380 emulator shall initialize SB to the highest available memory address + 1. **Note:** SB can be changed by a program at runtime.

Req3) Upon startup, and after reading the input file into prog_mem, the 4380 emulator shall initialize SP to the highest available memory address + 1. **Note:** SP can be changed by a program at runtime.

Req4) Upon startup, and after reading the input file into prog_mem, the 4380 emulator shall initialize SL to the address of the first byte past the last instruction read into program memory. **Note:** SL can be changed by a program at runtime.

Req5) The 4380 emulator shall not allow SP to be set to a value higher than SB, or lower than SL. If at any time a program attempts to set SP to a value outside the range of [SL, SB], the emulator shall print the message “INVALID INSTRUCTION AT: <address>” where <address> is the address in Program Memory of the first byte of the offending instruction. The emulator shall then exit with a return value of 1.

Req6) At runtime, SP shall store the address of the last/latest byte allocated on the stack. In other words, the next available byte on the stack shall be located at SP + 1.

Part 2: Programming the 4380

Now that you have a working 4380 Mark 4 - it's time to run it - and you - through your paces. For part 2 of this project you will write several assembly programs. To "warm you up" we will begin with simple programs, and demonstrate strategies you may find useful for writing assembly programs in general. As the programs progress they increase in difficulty/complexity, and we offer less direct aid. For the final program you are expected to be able to "stand on your own". You may discuss the first three programs with your classmates (but not share code), but the 4th and final program (Program D) should be considered an exam question and should be completed without any outside help (note: you are free to ask the instructor clarifying questions).

Program A: Fibonacci Sequence

Create a simple program that:

1. Prompts the user with the string "Please enter the Fibonacci term you would like computed: "
2. Reads an integer value from stdin
3. Computes the appropriate Fibonacci term (as indicated by the integer read from stdin) in the sequence beginning with initial values 0, 1. (i.e. 0, 1, 1, 2, 3...)
4. Prints the message "Term N in the Fibonacci sequence is: M\n" where N is the integer read in 2, and M is the value computed in 3.

To assist you in this process we have provided 3 files ProgA_0.cpp - ProgA_2.cpp which demonstrate a useful strategy for writing assembly programs. The strategy utilizes an incremental process that begins with solving the problem first in a language you are familiar

with (such as C or C++) and then incrementally modifying the program to "look and feel" increasingly like a program written in the target assembly language. At each stage of modification (and for as long as possible) you should attempt to keep the program "working" - which in the example provided means the source can be compiled and the resulting program behaves as desired. Continue the process until you reach a point at which you can "comfortably" write an assembly code version of the program.

Program B: A Modulus Function

Coming soon...

Program C: Recursive Fibonacci

Coming soon...

Program D: Prime Number Generator

Coming soon...

What to Turn In:

When you have completed the project you will need to submit the URL for your repository on the assignment page in Canvas. Your repository should be private **but shared with your course grader and/or instructor.** Instructions for granting access to your repo for the course grader will be provided by your instructor.

You must also submit source files (.asm) for programs A - D (submitted in Canvas).

Note: If you are invoking the Non-Late policy please insert a note at the top of your README file that states “INVOKING THE NON_LATE POLICY” followed immediately by your work log. (The rest of your README should still contain relevant project details including those discussed in previous projects).